

Taming optimization variance in compact neural shading networks

BENEDIKT BITTERLI, NVIDIA, USA
PETRIK CLARBERG, NVIDIA, Sweden
CHRIS CUMMINGS, NVIDIA, United Kingdom
AARON LEFOHN, NVIDIA, USA
STEVE MARSCHNER, Cornell University, USA and NVIDIA, USA
JAN NOVÁK, NVIDIA, Czech Republic
FABRICE ROUSSELLE, NVIDIA, Switzerland
ANDREA WEIDLICH, NVIDIA, Canada
TIZIAN ZELTNER, NVIDIA, Switzerland

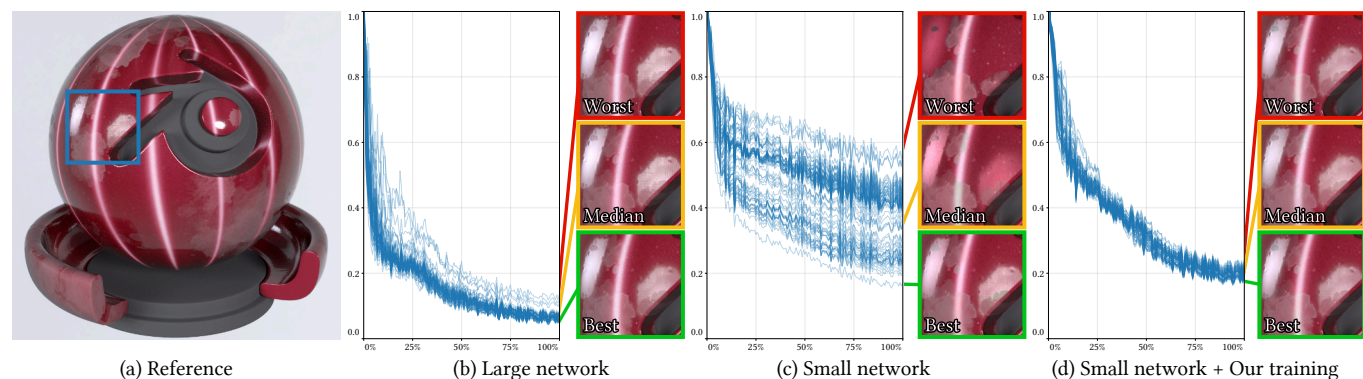


Fig. 1. We train 64 different initializations of a neural reflectance model for a challenging reference material with multiple layers (a). For a decoder network with excess capacity ((b), 54k parameters), all training runs converge to a solution of similar quality. For an undersized network barely able to capture the material ((c), 947 parameters), there is significant variance in training outcomes, with many instances missing important features. Using our multi-instance training approach (d), all runs converge to a solution of similar quality, at the same training cost as the baseline (c). Although the loss is higher than (b) as expected, training outcomes are equally as consistent as the much larger network.

We present a training algorithm to mitigate optimization instabilities in small neural networks, like those used in real-time neural shading applications. While large, overparameterized models exhibit predictable convergence, smaller architectures often suffer from high optimization variance: differently initialized models converge to disparate local minima. To reduce these training instabilities, we introduce an optimization approach that utilizes an ensemble of network instances during training. We prune underperforming instances and dynamically resize training batches to maintain wall-clock timings comparable to—or faster than—single-instance training. This strategy allows efficiently exploring the weight space yielding a well-performing

model with significantly higher likelihood than optimizing a single instance only. We develop and analyze the algorithm in the context of learning reflectance functions for neural shading. This is a challenging task for small neural models due to the high dynamic range of the target function. In addition to our multi-instance training method, we also revisit the choices of loss functions, activation functions, and input parameterization to further improve quality and training robustness.

CCS Concepts: • **Computing methodologies** → **Reflectance modeling; Neural networks.**

Additional Key Words and Phrases: Neural shading, Real-time rendering, Network training

ACM Reference Format:

Benedikt Bitterli, Petrik Clarberg, Chris Cummings, Aaron Lefohn, Steve Marschner, Jan Novák, Fabrice Rousselle, Andrea Weidlich, and Tizian Zeltner. 2026. Taming optimization variance in compact neural shading networks. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers (SIGGRAPH Conference Papers '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3799902.3811231>

1 Introduction

The recent popularity of *neural shading*—i.e. the use of neural networks within real-time rendering pipelines—has led to renewed

Authors' Contact Information: Benedikt Bitterli, NVIDIA, Redmond, USA, bbitterli@nvidia.com; Petrik Clarberg, NVIDIA, Lund, Sweden, pclarberg@nvidia.com; Chris Cummings, NVIDIA, Guildford, United Kingdom, ccummings@nvidia.com; Aaron Lefohn, NVIDIA, Redmond, USA, alefohn@nvidia.com; Steve Marschner, Cornell University, Ithaca, USA and NVIDIA, USA, smarschner@nvidia.com; Jan Novák, NVIDIA, Prague, Czech Republic, novakj4@gmail.com; Fabrice Rousselle, NVIDIA, Zurich, Switzerland, frousselle@nvidia.com; Andrea Weidlich, NVIDIA, Montreal, Canada, aweidlich@nvidia.com; Tizian Zeltner, NVIDIA, Zurich, Switzerland, tzeltner@nvidia.com.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Conference Papers '26, Los Angeles, CA, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2554-8/2026/07
<https://doi.org/10.1145/3799902.3811231>

interest in small neural networks as useful modeling primitives. We observe that most use cases (e.g. baking, compression) intentionally specialize small networks to the training data, and do not need to generalize to unseen data; it is more important to consistently find a good fit to the existing data.

Successful training of small models, however, poses different challenges than training the deep networks used in many other applications. For deep, overparametrized models, Frankle and Carbin [2019] suggest a high probability of the existence of a “lottery ticket”—a sub-network that achieves good performance after training. In small networks, with barely enough capacity for their task, however, the number of such subnetworks quickly diminishes, and “lottery tickets” become increasingly difficult to find.

The loss landscape of compact models is inherently more hostile and less smooth than with overparameterized models. As a result, models often converge to different local minima, making the optimization outcome highly sensitive to model initialization and training data ordering. This chaotic training performance hinders experimentation, development, and deployment of small networks in production, which can require thousands of shading networks to be trained repeatedly and reliably during asset development.

Our goal is to improve the robustness of small network training, so that any training run has a high probability of resulting in good performance. Training variability is a multi-faceted problem that involves the model architecture, loss, training data and optimization schedule. Although various strategies have been proposed—such as careful initialization [Glorot and Bengio 2010a; Mishkin and Matas 2016; Sutskever et al. 2013] or choice of activations [Lu et al. 2020]—we observe that, on their own, these do not suffice.

We introduce a training scheme that improves the reliability of small model training by a large factor. Inspired by methods for hyperparameter search such as *successive halving* [Jamieson and Talwalkar 2015; Li et al. 2018], we train many independent instances of the model in parallel and pick one of the best ones. Our analysis shows that the ranking of models from good to poor performers already emerges early in training, and we progressively cull the worst models while retaining the best model with high likelihood.

We pair this scheme with *batch size annealing* to further improve reliability while realizing large performance gains. The impact of the batch size on training has been studied extensively, and trades off exploration (small batches/noisy gradients) with exploitation (large batches/converged gradients) [Keskar et al. 2017]. This allows switching from a fixed batch size to a tailored schedule that gradually anneals from small batches that easily escape local minima early in training, to large batches that find narrow minimizers of the loss.

The novelty of our pipeline lies not in its building blocks (these have been explored in previous works) but in the codesign of the batch size and the instance schedule so that at each training step, a constant batch size is split evenly among the instances of the model. As poor performers are culled, their budget of training data is reallocated to the remaining instances. Our final method significantly reduces training variability at *identical training cost* compared to a single-instance baseline.

Enabled by this training scheme, we then perform a case study of a specific neural reflectance model [Zeltner et al. 2024] with challenging training dynamics to show how training stability can be

improved on a specific problem. We demonstrate how better choices of input parameterization, training loss, and activation function can have a large impact on training variability.

In summary, our contributions are: 1) a multi-instance training schedule that balances pruning and batch size annealing to maintain a fixed compute budget, and 2) an improved input parameterization, carefully selected training loss and activation function tailored to neural reflectance models. Our implementation is available at: <https://research.nvidia.com/labs/rtr/publication/bitterli2026taming>

2 Related work

Reviewing all approaches that are related to ours is beyond the scope of a single paper as the body of literature is vast. Instead, we focus on main concepts, listing a few examples for each.

Parallel training. Several approaches train multiple instances of the same model in parallel. For instance, *population-based training* [Doulazmi et al. 2025; Jaderberg et al. 2017; Li et al. 2019] utilizes a pool of instances, periodically overwriting underperformers using the weights and hyperparameters of better performing instances. These works bear similarity to our approach in that they also eliminate bad instances, however, they replace them with mutated copies of good performers keeping the population size constant. Deep ensembles [Fort et al. 2020] instead propose to retain all model instances and combine their outputs; inferring reflectance values using many networks, however, is not tractable for real-time applications.

Closer to our approach is *successive halving* [Jamieson and Talwalkar 2015; Li et al. 2018; Pham and Duh 2023; Zhang and Duh 2024] that eliminates worst performers, reducing the population size over time. The progressively fewer survivors are allocated more computational budget, typically by increasing the number of training iterations or epochs. In contrast, we propose to increase the batch size, keeping the iteration count constant.

Distillation and Pruning. Distillation techniques [Hinton et al. 2015] initially train a larger “teacher” model, which benefits from the stability of overparameterized networks, and use it to train a smaller “student” model ultimately used for inference. Another way to obtain a small network with good performance is to train a bigger model and then prune its weights or entire neurons. The structured pruning technique [Liu et al. 2017] deletes neurons with small contribution from the larger model. Valerio et al. [2022] follow a similar strategy, using freed memory from pruning to increase batch size. Our technique is similar in the batch size adjustment, but we operate on an ensemble of instances rather than a single one.

Batch size adjustment. Several works study the impact of changing batch size during training. Smith et al. [2018] show that increasing batch size can replicate the effect of learning rate decay, achieving similar training dynamics with fewer parameter updates. McCandlish et al. [2018] introduce the gradient noise scale, which typically grows during training and provides insights that can inform adaptive batch size schedules. AdaBatch [Devarakonda et al. 2017] adaptively increases batch size when learning stagnates, combining the convergence benefits of small batches with the efficiency of large batches.

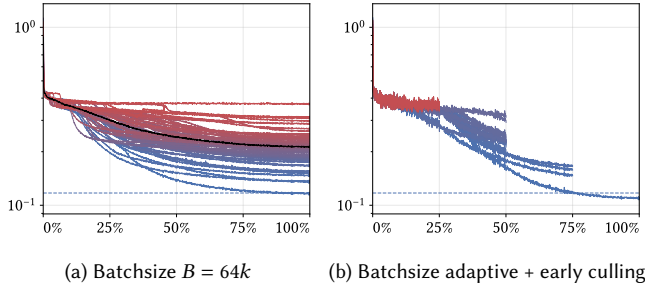


Fig. 2. (a) Loss curves of 64 different network instances. Curves are shaded based on rank when sorted by final loss from best (blue) to worst (red) instance. Even though curves cross occasionally, the relative ordering of instances emerges early in training and remains stable. (b) Our multi-instance approach with progressive batch sizes and instance culling. We perform $64\times$ fewer network evaluations than (a), but find the best instance regardless, while achieving slightly lower loss due to the induced batch schedule.

Neural reflectance. We address the problem of training robustness of small networks in the context of learning reflectance functions [Fan et al. 2022; Kuznetsov et al. 2019, 2021; Rainer et al. 2020, 2019; Sztrajman et al. 2021; Xue et al. 2024; Zeltner et al. 2024; Zheng et al. 2021]. Our experimental setup consists of optimizing a neural texture [Thies et al. 2019] and a small decoding MLP [Zeltner et al. 2024], which decodes the neural texture into the value of the bidirectional reflectance distribution function (BRDF) based on a local shading query. Our goal is to use the smallest MLP possible for high inference performance, facilitating adoption in real-time applications such as computer games.

3 Efficient multi-instance optimization

The loss landscape of a neural network contains many local minima, and which of these the network converges to depends highly on its initial weights. Different initializations of the same network may converge to very different weight configurations, and some variability in the training loss is unavoidable. However, if the model has sufficient overcapacity, we can still expect *all* of the configurations to perform similarly well (Figure 1 (b)). In contrast, smaller, under-powered models (Figure 1 (c)) suffer from a much larger spread of final loss values, and the performance of a trained model might vary significantly between different training runs.

In this section, we focus on a general strategy for improving training robustness for small models such that any training run has a high probability of resulting in good performance. Our method does this without increasing the training cost, allowing reliable and efficient training of neural shading networks (Figure 1d). The present section discusses the concepts behind our multi-instance scheme, the details of our case study setup will be detailed in Section 4.

Baseline approach. We measure training cost by the number of network evaluations, and our goal is to improve training stability without increasing this metric. To this end, we first define a baseline optimization approach, which trains a single network (detailed in Section 4) for N optimization steps. Each step evaluates a batch of

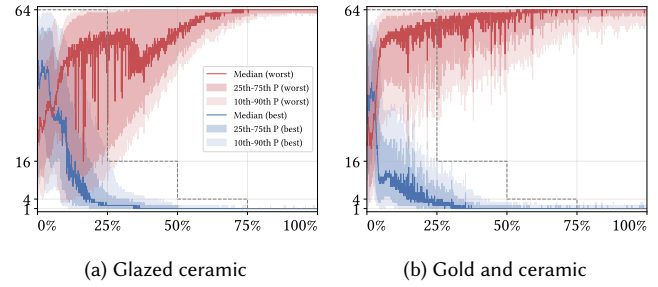


Fig. 3. Rank distribution of the best (blue) and worst (red) instance during training for 64-instances jobs. The lines indicate the median rank, the shaded areas show the 25th-75th and 10th-90th percentiles. The dashed staircase illustrates the culling threshold. We used constant batch size $B = 1k$.

training data of size B , for a total baseline cost of $N \times B$ network evaluations.

3.1 Simple multi-instance training

A simple strategy for improving training robustness is to train K independent instances of the model with different initializations and pick the best performer at the end. If the probability of training failure—i.e. unacceptable performance—is p , then this approach will reduce the chance of failure to p^K . However, this brute-force strategy also increases the training cost by a factor of K . To maintain the training cost of the baseline, either the number of steps or the batch size needs to be reduced by a factor of K , neither of which is a satisfactory solution.

3.2 Early instance culling

In Figure 2 (a), we plot loss curves of $K=64$ different instances of the baseline. Each loss curve is assigned a color based on its final rank (i.e. its index when sorted from best to worst loss at the end). Although individual curves cross at various points, we observe that the best performers at the end also rank among the best performers throughout most of training; analogously for the worst performers.

To demonstrate this directly, we show how the ranking of instances evolves over the course of training in Figure 3. We again train $K = 64$ instances, identify the best and worst instance after the last training step, and track their ranking relative to the other instances at all prior training steps. To exclude random noise, we repeat this process 100 times with different initializations and compute the median rank and rank variance of the best/worst performers.

Although initially all instances perform similarly (poorly), the gap between the best and worst instance grows rapidly. After an initial warm-up phase, the best instance reliably ranks among the top performers. This suggests that not all instances have to be trained to convergence: Bad models are unlikely to suddenly rank among the best, and their training can be terminated early.

The rank of the best performer roughly follows an exponential profile. Based on this observation, we propose the following progressive training scheme:

- (1) Initialize K independent instances of the model.
- (2) Divide the N optimization steps into P phases.



Fig. 4. We use two sets of test objects. **Left:** The da Vinci objects were chosen to capture a wide range of reflectance behaviors, ranging from perfectly specular to almost diffuse. **Right:** The multi-layered materials are more challenging to train due to strongly view-dependent effects and multiple normal maps.

- (3) Train each instance for N/P steps.
- (4) Cull the number of instances by a factor of $K^{1/(P-1)}$: Order them by loss and only retain the best performers at that step.
- (5) Repeat steps (3) and (4) until one instance remains.

This achieves similar performance as the much more expensive brute force approach, but now at a cost of roughly $K/\log(K)^1$ times the cost of the baseline.

3.3 Batch size annealing

With early culling of instances, the cost is now dominated by the early training phases: With $K=64$ instances and $P=4$ phases, the cost of the first phase alone is $K/P = 16\times$ the entire training cost of the baseline.

There are not many options to reduce this cost without also impairing performance: If we reduce the length of the initial phase and cull earlier, we remove instances before the ranks of good and bad performers are well separated and risk discarding the best instances. If we reduce the number of initial instances K instead, we reduce the chance of finding a good performer.

The only remaining choice is to adjust the *batch size*. To maintain constant training cost regardless of the number of instances, we equally divide a fixed budget of training data among all instances. Early in training, we train many instances but give each only a small slice of the budget by using small batch sizes; then, as training progresses, the budget is redistributed every time instances are culled. Our final training scheme is illustrated in Figure 2 (b) and is identical to that of Section 3.2, with one change:

- (3) Train each instance for N/P steps, and at each step, divide a fixed budget of training data among the active instances.

This approach ends up being quite similar to approaches in hyperparameter search like *successive halving* [Jamieson and Talwalkar 2015; Li et al. 2018], which also progressively cull parallel training runs and redistribute their budget. A crucial difference to these works is that they divide a budget of training steps among instances, which would start culling much too early in our case and lead to a removal of potentially good instances.

Shared by all these approaches however is the inherent advantage that all phases of training now have equal cost. In fact, the total

¹Lower bound obtained by letting $P \rightarrow \infty$. Finite P achieve slightly lower efficiency.

number of network evaluations of our approach is *identical* to that of the baseline: The only difference is how the budget of evaluations is allocated.

Discussion. Reducing the batch size can potentially harm the convergence behavior, and this change needs careful evaluation. The size of the training batch determines the variance of the parameter gradients, which influences what kinds of minima training will converge to: Narrow minima require gradients with low noise levels, whereas the noisy gradients of smaller batches preferably converge to wide minima [Keskar et al. 2017].

Dividing the training batch among the active instances effectively acts as a batch size schedule. This *annealing* of the batch size has been shown to behave similarly to the more commonly used learning rate decay [Smith et al. 2018] and actually proves beneficial: Early in training, models are more prone to getting “stuck” in bad local minima, which smaller batches help to avoid. The opposite is true at the end of training, where larger batches are able to find the bottom of narrow loss valleys. We confirm this with empirical results next.

4 Analysis

In this section, we compare the performance of the proposed algorithm against a number of non-culling multi-instance configurations. We also analyze how often the algorithm (incorrectly) culls well performing instances, and how much the retained instance differs from the best one in terms of the loss function.

Optimization setup. We use the same setup as Zeltner et al. [2024] for learning spatially varying BRDFs (SVBRDFs). A single training record consists of:

- *shading configuration:* surface location $\mathbf{x}$ together with incident and outgoing directions ω_i and ω_o in the local frame,
- *surface attributes:* albedo, normal, tangent, roughness, and weight at $\mathbf{x}$ (for each layer of the material), and
- reference BRDF value $f(\mathbf{x}, \omega_i, \omega_o)$.

We train an encoder-decoder pair that encodes the surface attributes into a latent code, which are then decoded, conditioned on ω_i and ω_o , to yield an approximate BRDF value $\hat{f}(\mathbf{x}, \omega_i, \omega_o)$. Following Zeltner et al., we transform ω_i and ω_o into two learned shading frames to better resolve normal-mapped details. After training, only the

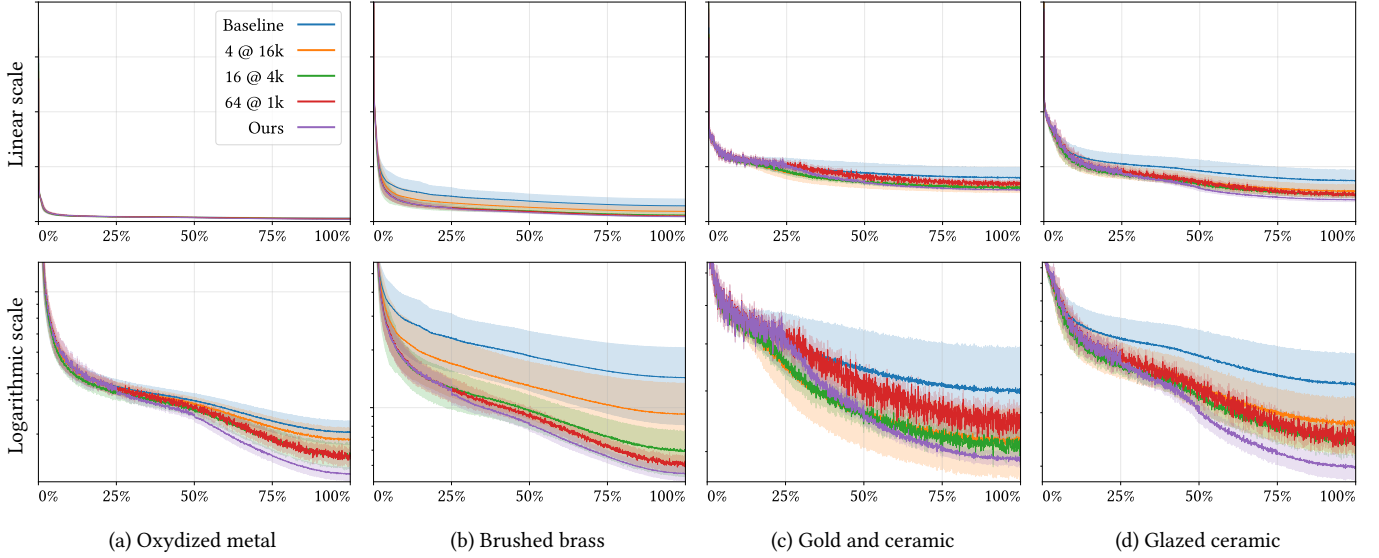


Fig. 5. We compare the single-instance, baseline approach (blue) against three *simple* multi-instance configurations, which split the compute budget of the baseline between K instances by varying the batch size B (these are labeled “ $K @ B$ ” in the legend). We also include our multi-instance approach that culls instances and anneals the batch size (purple) to, again, match the cost of the baseline. All techniques are compared on four materials, ordered by visual complexity from left to right. The **top** row shows the loss functions using a linear scale (and all plots have the same range), to highlight that the left materials are easier to learn (lower absolute loss). The **bottom** row shows the same losses but with a logarithmic scale to allow close inspection of relative performance.

latent code and decoder are kept for inference. The weights of the model are updated using the Adam optimizer [Kingma and Ba 2014] according to an objective loss function $\mathcal{L}(\hat{f}(\cdot), f(\cdot))$. Although we tested other optimizers, we found them to produce a similar spread in training outcomes as Adam (see supplemental material), and we use Adam for all results. The trained instances differ only in the seed used for generating the initial random weights; we use Glorot initialization [Glorot and Bengio 2010b] in all our results.

Materials. We use the two sets of materials shown in Figure 4. The first set is created from 9 objects taken from the publicly available da Vinci Workshop dataset [NVIDIA 2023]. For each object, we unwrap it into a single UV tile and bake 4K texture maps to produce a single reference SVBRDF per object, which serves as the training target for the neural model. Finally, we translate the MDL materials into a USDPreviewSurface [Pixar 2019].

The second set consists of six high-quality multi-layered materials. All materials of this set consist of a base material and several top layers, simulating effects like e.g., glazing, stains, or dust. Each layer has its own set of parameters and textures, including multiple normal maps. Combined with high specularities, this makes them especially challenging to represent with compact models.

Hyperparameters. For the rest of this section, we use our training scheme with a schedule of 64 initial instances and $P = 4$ training phases. The number of instances is culled by a factor of 4 at the end of each phase, and the batch size grows by the same factor, with a progression of $K = 64, 16, 4, 1$ instances and $B = 1k, 4k, 16k, 64k$ respectively. All remaining hyperparameters (learning rate, momentum, etc.) are identical across all methods and materials. For all

results, we use the same network architecture with three hidden 64-wide layers for the encoder and two hidden 16-wide layers for the decoder.²

Our approach vs. simpler configurations. At the beginning of Section 3, we defined a *baseline* method, which optimizes a single network in N steps with batch size B , i.e., using $N \times B$ network evaluations. Here we consider a few *simple* multi-instance configurations, which trade the size of the batch for number of instances, against our approach. We summarize these configurations in the following table:

	Num. of instances K	Batch size B
Baseline	1	64k
4 @ 16k	4	16k
16 @ 4k	16	4k
64 @ 1k	64	1k
Ours	64 → 16 → 4 → 1	1k → 4k → 16k → 64k

All compared techniques are trained using $N = 100k$ optimization steps, and perform exactly the same number of network evaluations.

Figure 5 shows the loss curves of individual techniques on four different materials with increasing level of visual complexity (from left to right). Each curve represents the mean loss over 64 independent optimization runs; the band around the curve depicts one standard deviation.

²In practice, the impact of our multi-instance training diminishes with increasing network sizes that tend to converge reliably (see Figure 1b). Nevertheless, we always keep it enabled as it does not hurt and occasionally improves results.

Table 1. Relative frequency that the surviving instance (selected by culling) is among the best performing instances. The first column shows the probability that it is the best instance (rank 1), the next columns show the probability that the surviving instance is among the top 3 (top 5%), top 6 (top 10%), and top 12 (top 20%) of all instances.

Material	Best (rank 1)	Top 5% (top 3)	Top 10% (top 6)	Top 20% (top 12)
DA VINCI WORKSHOP MATERIALS				
Birdcage	93.3%	100.0%	100.0%	100.0%
Chandelier	73.3%	80.0%	93.3%	100.0%
Hammer	86.7%	100.0%	100.0%	100.0%
Lantern	86.7%	100.0%	100.0%	100.0%
Mirror	60.0%	93.3%	100.0%	100.0%
Palette	66.7%	93.3%	100.0%	100.0%
Chair	93.3%	100.0%	100.0%	100.0%
Scales	80.0%	100.0%	100.0%	100.0%
Table	60.0%	80.0%	86.7%	93.3%
LAYERED MATERIALS				
Scratched steel	100.0%	100.0%	100.0%	100.0%
Bumpy plastic	93.3%	100.0%	100.0%	100.0%
Oxydized metal	66.7%	100.0%	100.0%	100.0%
Gold and ceramic	66.7%	93.3%	100.0%	100.0%
Brushed brass	86.7%	93.3%	100.0%	100.0%
Glazed ceramic	73.3%	100.0%	100.0%	100.0%
Average	79.1%	95.6%	98.7%	99.6%

For simple materials, such as the rough oxydized metal (column a), all configurations perform robustly, with multi-instance training yielding marginally better results (cf. the bottom plot with the logarithmic y-axis).

For complex, layered materials, the performance of different configurations vary more dramatically. In most cases (column a, b, and d), increasing the number of instances (while decreasing the batch size) tends to improve results by performing a more thorough exploration of the configuration space. However, noisy gradients due to small batches also prevent the optimization from settling in a low-loss configuration.

Our multi-instance approach with culling and annealing transitions from an early exploration to later exploration of surviving instances, yielding robust performance, and sometimes improving upon the second best configuration by a wide margin. In the supplemental material, we provide results with other culling schedules and batch size annealing.

Retention probability. Given that our culling algorithm is greedy, there is a non-zero chance that it may cull away the best or even all the good performers. In Table 1, we analyze the retention probability, i.e., the probability that we retain the best instance, or one of the top 5% or top 10% performers. We carry out the analysis over 24 optimization trials per material and report mean probabilities for each material, as well as over all 360 trials.

Averaged over all optimization trials, the empirical probability of retaining the best instance is 74.4%, and retaining one of the top 10% (top 6) performers is 99.4%. This means that we did not always

Table 2. We disable culling, and merely simulate it to identify the instance that would survive. We report the final loss of the best instance across all 64 instances (**2nd column**) and the loss of *our* surviving instance (normalized by the value of the best) (**3rd column**). The last column shows the same ratio but for a *randomly* picked instance.

Material	Best loss $\mathcal{L}_b$	Our inst. $\mathcal{L}_o/\mathcal{L}_b$	Random inst. $\mathcal{L}_r/\mathcal{L}_b$
DA VINCI WORKSHOP MATERIALS			
Birdcage	0.054 ± 0.003	1.001 ± 0.002	1.872 ± 0.120
Chandelier	0.038 ± 0.003	1.024 ± 0.060	1.814 ± 0.123
Hammer	0.032 ± 0.003	1.003 ± 0.009	1.686 ± 0.130
Lantern	0.022 ± 0.002	1.001 ± 0.003	1.769 ± 0.148
Mirror	0.081 ± 0.005	1.010 ± 0.017	1.808 ± 0.120
Palette	0.260 ± 0.014	1.006 ± 0.016	1.576 ± 0.091
Chair	0.044 ± 0.005	1.000 ± 0.001	3.279 ± 0.473
Scales	0.046 ± 0.002	1.005 ± 0.015	1.704 ± 0.071
Table	0.019 ± 0.001	1.067 ± 0.131	1.558 ± 0.117
LAYERED MATERIALS			
Scratched steel	0.056 ± 0.005	1.000 ± 0.000	2.450 ± 0.235
Bumpy plastic	0.018 ± 0.001	1.001 ± 0.005	1.360 ± 0.062
Oxydized metal	0.022 ± 0.001	1.004 ± 0.006	1.416 ± 0.065
Gold and ceramic	0.283 ± 0.007	1.005 ± 0.013	1.444 ± 0.063
Brushed brass	0.045 ± 0.004	1.015 ± 0.041	3.575 ± 0.303
Glazed ceramic	0.201 ± 0.021	1.009 ± 0.018	1.870 ± 0.198
Average	0.081 ± 0.089	1.010 ± 0.017	1.946 ± 0.658

retain the best performer, but almost always produced one of the top performers.

Loss value analysis. We also study the difference between the final losses of the surviving instance, compared to the best performing instance. In Table 2, we list the final loss value $\mathcal{L}_b$ of the best performer, followed by the loss of the surviving instance $\mathcal{L}_o$ expressed in relative terms as $\mathcal{L}_o/\mathcal{L}_b$. A value of 1 one means the surviving instance performed the same as the best one (i.e., it likely is the best one), a value of 2 amounts to 2× higher loss, and so forth. We also report the same relative loss for a randomly selected instance, which corresponds to the performance of the baseline. All listed numbers are mean values and standard deviations over 15 optimization trials.

In terms of the loss function, our culling algorithm retains an instance that always performs very close to the best performer; the $\mathcal{L}_o/\mathcal{L}_b$ ratio is always close to 1. In contrast, a random instance from the ensemble yields much higher ratio, especially for some of the layered materials (e.g., scratched steel, brushed brass, and glazed ceramic).

Runtime performance. Industry adoption of neural materials not only requires dependable training, but also fast training in order to enable quick iteration by lookdev artists, and allow retraining the hundreds of materials that are typically used in a production scene. All algorithms in this article are implemented in SlangPy [Kallweit et al. 2025] as fully fused kernels, where the number of networks trained in parallel has marginal impact on training performance. As such our technique has a comparable wall-clock timings to the baseline. Because the initial weights have a much higher impact on training variation than the training data, we can further optimize

Table 3. Optimization timings (in seconds) of the baseline method and our multi-instance technique with culling and batchsize annealing. The latter is implemented to share training data between instances, which reduces the amount needed and leads to faster processing times. The timings were measured on a single NVIDIA RTX 5090 GPU.

Material	Baseline (1 @ 64k)	Ours
Oxydized metal	86.50 s	59.00 s
Gold and ceramic	78.05 s	55.80 s
Glazed ceramic	81.02 s	57.11 s
Brushed brass	79.48 s	54.40 s

training by using the same training data for all instances. This reduces the amount of data needed in the first, second, and third phase by a factor of $64\times$, $16\times$, and $4\times$, respectively, which amounts to an overall $3.01\times$ reduction in training data. In the supplemental material, we present results indicating that, on average, data sharing helps with instance ranking, without impacting the quality of the best performing instance.

In Table 3 we provide wall-clock timings (in seconds) of the baseline method and our optimized implementation of the multi-instance technique with culling and annealing, measured on a single NVIDIA RTX 5090 GPU. The reduction in training data manifests in 28–32% faster training times compared to the baseline. Implementations in other frameworks may incur different overheads.

5 Additional improvements for neural materials

In the previous sections, we proposed and analyzed our primary contribution: a multi-instance optimization approach with pruning and batchsize scaling. In this section, we discuss additional improvements for robustly training neural appearance models.

A quantitative comparison on validation image errors can be found in the supplemental material.

Input parameterization. The parameterization of ω_i and ω_o has a strong influence on the model performance [Xu et al. 2025]. Prior works have input the directions either directly (e.g., in Zeltner et al. [2024]) or expressed as half and difference vectors ω_h and ω_d [Rusinkiewicz 1998] (e.g., in Sztrajman et al. [2021]).

The latter can improve the representation of specular peaks, especially at grazing angles. However, the original formulation contains a singularity in ω_d (Figure 6, top row) and leads to unstable training:

$$\omega_d = R_y(-\theta_h)R_z(-\phi_h)\omega_i \quad (1)$$

At the point of perfect reflection, the half vector ω_h aligns with the surface normal and the azimuth angle ϕ_h is undefined.

We found a more stable difference vector formulation (Figure 6, bottom row) using the shortest-arc rotation that aligns ω_h with the z -axis $[0, 0, 1]^T$:

$$\omega_d' = \omega_i h_z + (\mathbf{v} \times \omega_i) \frac{(\mathbf{v} \cdot \omega_i) \mathbf{v}}{1 + h_z}. \quad (2)$$

In practice, we also observed cases where the half/difference parameterization becomes counterproductive, e.g. in presence of normal mapping which may rotate the BRDF lobe away from its expected alignment. To mitigate this, we provide the network with the

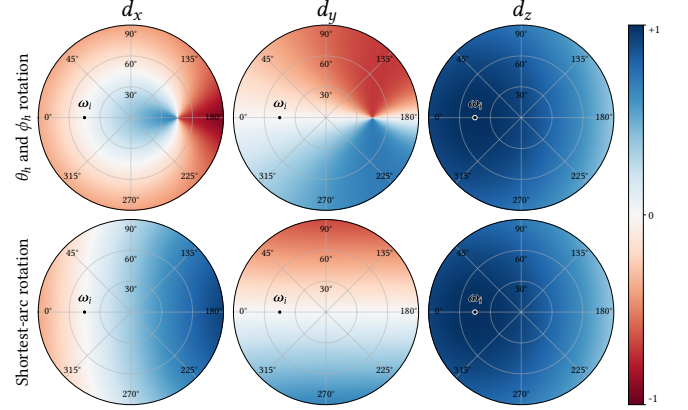


Fig. 6. False-color visualization of the three components (d_x, d_y, d_z) of the difference vector as a function of the outgoing direction ω_o (projected on the unit disk) for a fixed incident direction ω_i (black dot). **Top:** The formulation based on θ_h/ϕ_h rotations [Rusinkiewicz 1998] exhibits a singularity at the point of perfect reflection. For this distorted mapping, rapidly changing input values must represent nearly identical BRDF values, hindering optimization. **Bottom:** The shortest-arc rotation provides a smooth parameterization on the entire hemisphere that is easier to resolve.

full set of vectors ($\omega_i, \omega_o, \omega_h, \omega_d'$), allowing the model to adaptively prioritize the most suitable parameterization for a given material.

Loss function. To predict non-negative, high dynamic range BRDF values, we follow prior work [Sztrajman et al. 2021] and apply an exponential mapping $\hat{f} = \exp(z)$ to the MLP output z . For training, we use a mapped L_1 loss:

$$\mathcal{L} = \left| \mathcal{M}(f) - \mathcal{M}(\hat{f}) \right|, \quad (3)$$

where $\mathcal{M}$ is a non-linear function. A popular choice is the logarithmic mapping $\mathcal{M}_{\log}(x) = \log(1 + x)$ [Kuznetsov et al. 2019, 2021; Rainer et al. 2020, 2019; Sztrajman et al. 2021; Zeltner et al. 2024]. However, we found it to struggle at low target values, failing to capture diffuse details.

This issue stems from vanishing gradients with respect to z :

$$\left| \frac{\partial \mathcal{L}}{\partial z} \right| = \left| \mathcal{M}'(\hat{f}) \cdot \frac{d \exp(z)}{dz} \right| = \mathcal{M}'(\hat{f}) \cdot \hat{f} \quad (4)$$

As shown in Figure 7 (blue curve), $\mathcal{M}_{\log}$ yields a sigmoid-shaped gradient $\frac{\exp(z)}{1+\exp(z)}$ that suffers from near-zero gradients at small values (diffuse regions) and saturates at 1 for large values (peaks).

We propose a power mapping $\mathcal{M}_{\text{pow}}(x) = n(x^{1/n} - 1)$ with $n = 3$ instead. The resulting gradient $\left| \frac{\partial \mathcal{L}_{\text{pow}}}{\partial z} \right| = \exp(z/n)$ strikes a better balance across magnitudes: it remains non-zero in diffuse regions and continues growing for increasing outputs z , see the red curve in Figure 7. Figure 8 illustrates how this can result in visual improvements. Note that, as n increases, the power mapping converges to the natural logarithm mapping. In other words, the value of n allows to interpolate between a linear and a logarithmic mapping. Xue et al. [2024] proposed a closely related loss using a root of $n = 4$.

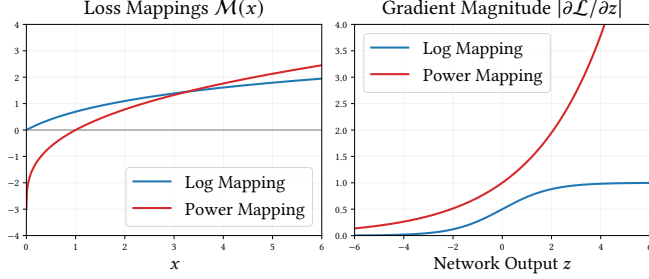


Fig. 7. **Left:** The logarithmic mapping $M_{\log}$ and the proposed power mapping M_{pow} . **Right:** Corresponding gradient magnitudes $|\partial\mathcal{L}/\partial z|$. While $M_{\log}$ leads to vanishing gradients for small values and saturation for large values, M_{pow} maintains gradients across the entire dynamic range.

Activation function. The standard ReLU activation is often sub-optimal for modeling BRDFs. It is prone to dying gradients, which is particularly detrimental in the compact architectures used for real-time neural materials, where inactive neurons can severely bottleneck capacity. Additionally, ReLU’s piecewise linearity produces visible facets in specular peaks, where discontinuities are most apparent (Figure 9).

To address this, we adopt Smooth ReLU (SmeLU) [Shamir et al. 2020], which ensures C^1 continuity via a quadratic transition region for $|x| < \beta$. We utilize a “Leaky” variant to maintain robust gradient flow, adding a small slope ϵ to the inactive region ($x \leq -\beta$):

$$\text{LeakySmeLU}(x; \beta, \epsilon) = \begin{cases} \epsilon x & x \leq -\beta \\ (1 - \epsilon) \frac{(x + \beta)^2}{4\beta} + \epsilon x & |x| < \beta \\ x & x \geq \beta \end{cases} \quad (5)$$

While standard LeakyReLU can sometimes achieve lower training losses, LeakySmeLU is a suitable replacement when the faceted appearance of specular highlights must be avoided.

6 Discussion and Conclusion

In this work, we introduce an efficient multi-instance training approach for small networks, motivated by their inherent optimization instabilities. By leveraging an ensemble of concurrently trained instances, carefully culling poor ones, and adjusting batch sizes, our method reduces optimization variance without increasing training cost. This allows us to reliably find well performing models. We apply the method to learning high-dynamic-range reflectance functions, and additionally propose a new parameterization of directions, the use of a smooth activation function, and a new loss function to further improve model and training performance. Our approach facilitates the industry adoption of small neural shading networks in real-time applications, where inference speed is crucial and fast and reliable training is mandatory. While our work focused on finding the best candidate among multiple instances, it could be combined with techniques focused on improving the candidates themselves. In that regard, integrating stochastic weight averaging [Izmailov et al. 2018; Madhyastha and Jain 2019] into our pipeline is a promising avenue for exploration, as the methodology is orthogonal to ours.



Fig. 8. Visual impact of the loss mappings. The logarithmic mapping (**left**) works well on the specular peaks, but ignores inaccuracies in the diffuse regions (stripes, stains, etc.). The power mapping (**right**) balances these very large and very small BRDF values more gracefully. Insets show HDR 7LIP [Andersson et al. 2021] error maps compared to the reference (**center**).

Acknowledgments

We want to thank Craig Kolb, Simon Kallweit and Tomas Davidovic for development of the supporting software stack, and Theresa Foley, Yong He and Sai Bangaru for evolving the Slang language that runs this project. We also thank Milos Hasan, Jaakko Lehtinen, Yaobin Ouyang and Wessam AlBahassi for their valuable input. The material test blob in Figure 1, 4 and 8 was created by Robin Marin and released under CC (<https://creativecommons.org/licenses/by/3.0/>)

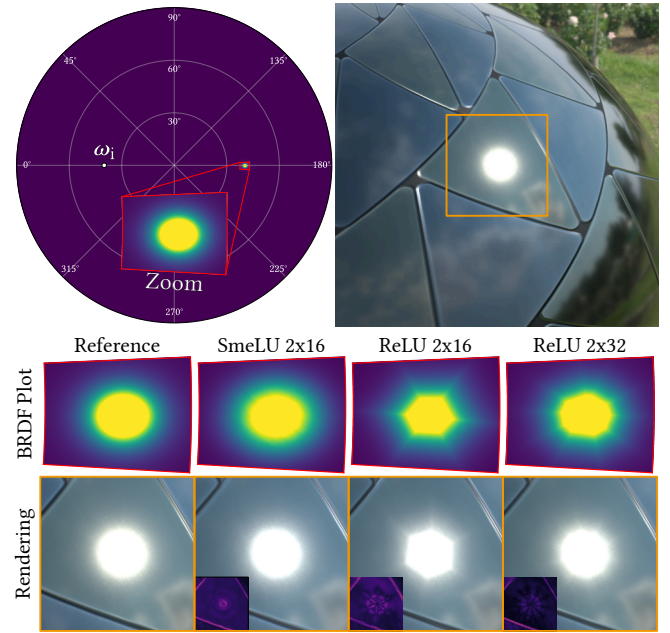


Fig. 9. **Top:** Projected hemisphere polar plots and rendering of a specular material. **Bottom:** Zoomed insets of plots and renderings for corresponding neural materials with different model size and activations. Small models (e.g. 2 hidden layers of 16 neurons) with a ReLU-based activation exhibit faceting artifacts in sharp highlights that require more modeling capacity (e.g. 2 hidden layers of 32 neurons) to smooth out. SmeLU-based activations provide a smoother reconstruction even at small model sizes. The small insets in the renderings show HDR 7LIP [Andersson et al. 2021] error maps.

References

- Pontus Andersson, Jim Nilsson, Peter Shirley, and Tomas Akenine-Möller. 2021. Visualizing Errors in Rendered High Dynamic Range Images. In *Eurographics Short Papers*. 25–28. doi:10.2312/egs.20211015
- Aditya Devarakonda, Maxim Naumov, and Michael Garland. 2017. AdaBatch: Adaptive Batch Sizes for Training Deep Neural Networks. In *Advances in Neural Information Processing Systems Workshops*. <https://arxiv.org/abs/1712.02029>
- Wael Doulazmi, Auguste Lehuger, Marin Toromanoff, Valentin Charraut, Thibault Buhet, and Fabien Moutarde. 2025. Multiple-Frequencies Population-Based Training. *CoRR* abs/2506.03225 (2025). arXiv:2506.03225 [cs.LG] <http://arxiv.org/abs/2506.03225>
- Jiahui Fan, Beibei Wang, Miloš Hašan, Jian Yang, and Ling-Qi Yan. 2022. Neural Layered BRDFs. In *ACM SIGGRAPH 2022 Conference Proceedings*. Association for Computing Machinery, Article 4. doi:10.1145/3528233.3530732
- Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. 2020. Deep Ensembles: A Loss Landscape Perspective. arXiv:1912.02757 [stat.ML] <https://arxiv.org/abs/1912.02757>
- Jonathan Frankle and Michael Carbin. 2019. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rjl-b3RcF7>
- Xavier Glorot and Yoshua Bengio. 2010a. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 9)*. PMLR, 249–256. <https://proceedings.mlr.press/v9/glorot10a.html>
- Xavier Glorot and Yoshua Bengio. 2010b. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 9)*. Yee Whye Teh and Mike Titterton (Eds.). PMLR, Chia Laguna Resort, Sardinia, Italy, 249–256. <https://proceedings.mlr.press/v9/glorot10a.html>
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML] <https://arxiv.org/abs/1503.02531>
- Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. 2018. Averaging Weights Leads to Wider Optima and Better Generalization. In *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI)*. AUAI Press, 876–885. doi:10.48550/arXiv.1803.05407
- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. 2017. Population Based Training of Neural Networks. *CoRR* abs/1711.09846 (2017). arXiv:1711.09846 [cs.LG] <http://arxiv.org/abs/1711.09846>
- Kevin Jamieson and Ameet Talwalkar. 2015. Non-stochastic Best Arm Identification and Hyperparameter Optimization. arXiv:1502.07943 [cs.LG] <https://arxiv.org/abs/1502.07943>
- Simon Kallweit, Chris Cummings, Benedikt Bitterli, Sai Bangaru, and Yong He. 2025. *SlangPy*. <https://github.com/shader-slang/slangpy>.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. 2017. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. arXiv:1609.04836 [cs.LG] <https://arxiv.org/abs/1609.04836>
- Diederik Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations* (12 2014).
- Alexandr Kuznetsov, Miloš Hašan, Zexiang Xu, Ling-Qi Yan, Bruce Walter, Nima Khademi Kalantari, Steve Marschner, and Ravi Ramamoorthi. 2019. Learning generative models for rendering specular microgeometry. *ACM Trans. Graph.* 38, 6, Article 225 (Nov 2019). doi:10.1145/3355089.3356525
- Alexandr Kuznetsov, Krishna Mullia, Zexiang Xu, Miloš Hašan, and Ravi Ramamoorthi. 2021. NeuMIP: multi-resolution neural materials. *ACM Trans. Graph.* 40, 4, Article 175 (Jul 2021). doi:10.1145/3450626.3459795
- Ang Li, Ola Spyra, Sagi Perel, Valentin Dalibard, Max Jaderberg, Chenjie Gu, David Budden, Tim Harley, and Pramod Gupta. 2019. A Generalized Framework for Population Based Training. In *Proc. of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1791–1799. doi:10.1145/3292500.3330649
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *Journal of Machine Learning Research* 18, 185 (2018), 1–52. <http://jmlr.org/papers/v18/li16-558.html>
- Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. 2017. Learning Efficient Convolutional Networks through Network Slimming. arXiv:1708.06519 [cs.CV] <https://arxiv.org/abs/1708.06519>
- Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. 2020. Dying ReLU and Initialization: Theory and Numerical Examples. *Communications in Computational Physics* 28, 5 (June 2020), 1671–1706. doi:10.4208/cicp.0a-2020-0165
- Pranava Madhyastha and Rishabh Jain. 2019. On Model Stability as a Function of Random Seed. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*. Mohit Bansal and Aline Villavicencio (Eds.). Association for Computational Linguistics, Hong Kong, China, 929–939. doi:10.18653/v1/K19-1087
- Sam McCandlish, Jared Kaplan, Dario Amodei, and Tom B. Brown. 2018. An Empirical Model of Large-Batch Training. *CoRR* abs/1812.06162 (2018). arXiv:1812.06162 [cs.LG] <https://arxiv.org/abs/1812.06162>
- D. Mishkin and J. Matas. 2016. All you need is a good init. In *Proceedings of ICLR*.
- NVIDIA. 2023. da Vinci’s Workshop. https://docs.omniverse.nvidia.com/USD/latest/USD_Content_Samples/davinci_workshop.html#da-vinci-s-workshop
- Nghia Pham and Kevin Duh. 2023. A Hyperparameter Optimization Toolkit for Neural Machine Translation Research. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, Toronto, Canada.
- Pixar. 2019. USDPreviewSurface. https://openUSD.org/release/spec_USDPreviewSurface.html
- Gilles Rainer, Abhijeet Ghosh, Wenzel Jakob, and Tim Weyrich. 2020. Unified Neural Encoding of BTFs. *Computer Graphics Forum* 39, 2, 167–178. doi:10.1111/cgf.13921
- Gilles Rainer, Wenzel Jakob, Abhijeet Ghosh, and Tim Weyrich. 2019. Neural BTF Compression and Interpolation. *Computer Graphics Forum* 38, 2, 235–244. doi:10.1111/cgf.13633
- Szymon Rusinkiewicz. 1998. A New Change of Variables for Efficient BRDF Representation. In *Rendering Techniques ’98*. Springer Vienna, Vienna, 11–22.
- Gil I. Shamir, Dong Lin, and Lorenzo Coviello. 2020. Smooth activations and reproducibility in deep networks. arXiv:2010.09931 [cs.LG] <https://arxiv.org/abs/2010.09931>
- Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. 2018. Don’t Decay the Learning Rate, Increase the Batch Size. arXiv:1711.00489 [cs.LG] <https://arxiv.org/abs/1711.00489>
- Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. 2013. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*. PMLR, 1139–1147.
- Alejandro Sotrajman, Gilles Rainer, Tobias Ritschel, and Tim Weyrich. 2021. Neural BRDF Representation and Importance Sampling. *Computer Graphics Forum* 40, 6, 332–346. doi:10.1111/cgf.14335
- Justus Thies, Michael Zollhöfer, and Matthias Nießner. 2019. Deferred neural rendering: image synthesis using neural textures. *ACM Trans. Graph.* 38, 4, Article 66 (Jul 2019). doi:10.1145/3306346.3323035
- Lorenzo Valerio, Franco Maria Nardini, Andrea Passarella, and Raffaele Perego. 2022. Dynamic Hard Pruning of Neural Networks at the Edge of the Internet. *Journal of Network and Computer Applications* 200 (2022). doi:10.1016/j.jnca.2021.103330
- Zilin Xu, Yehonathan Litman, Lingqi Yan, Yang Zhou, and Anton Michels. 2025. *Improving Angular Parameterization for Compact Neural Materials*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3757374.3771447>
- Bowen Xue, Shuang Zhao, Henrik Wann Jensen, and Zahra Montazeri. 2024. A Hierarchical Architecture for Neural Materials. *Computer Graphics Forum* 43, 6 (2024), e15116. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.15116 doi:10.1111/cgf.15116
- Tizian Zeltner, Fabrice Rousselle, Andrea Weidlich, Petrik Clarberg, Jan Novák, Benedikt Bitterli, Alex Evans, Tomáš Davidovič, Simon Kallweit, and Aaron Lefohn. 2024. Real-Time Neural Appearance Models. *ACM Trans. Graph.* 43, 3, Article 33 (Jun 2024). doi:10.1145/3659577
- Xuan Zhang and Kevin Duh. 2024. Best Practices of Successive Halving on Neural Machine Translation and Large Language Models. In *Proc. of the 16th Conference of the Association for Machine Translation in the Americas (Volume 1)*. 130–139.
- Chuanqun Zheng, Ruzhang Zheng, Rui Wang, Shuang Zhao, and Hujun Bao. 2021. A Compact Representation of Measured BRDFs Using Neural Processes. *ACM Trans. Graph.* 41, 2, Article 14 (Nov 2021). doi:10.1145/3490385